CCS2300 · WEEK 10 PROGRESS VIDEO

Railway Management System Six-Member Full-Mark Presentation Guide

Exact 8–10 minute plan, scripts, demonstrations, code ownership, complexity, testing, viva preparation and recording checklist

---

Project: Data Structures-Based Smart Railway Management System

Group size: Exactly 6 members

Recommended final duration: 9 minutes 15 seconds

Prepared against: CCS2300 Group Assignment Guidelines and the current Java source code

Date: 27 July 2026

1. Important truth before using this guide

This guide is designed to cover every progress-video marking category. It cannot guarantee a mark because grading depends on the actual implementation, visible evidence, delivery and answers. Never claim a feature works unless it can be executed or supported by source/test evidence.

Current tree gap: the assignment requires BST and AVL *insert, delete, search, inorder, preorder and postorder*. The current small `TrainBST` and `TrainAVLTree` implement insert, search and inorder; AVL also has balancing rotations. They do not yet implement delete, preorder or postorder, and neither tree is exposed through the running menu. For the strongest progress demonstration, complete and test these missing tree operations before recording. If they remain incomplete, state the exact current status and future plan honestly.

Sorting is not required in the Week 10 progress demonstration. The brief says searching and sorting introduced in Weeks 11–12 are for the final submission. Mention Bubble, Selection, Insertion, Merge and Quick Sort only in the future-development plan. Linear search is already present in the train linked list.

Marking alignment

Criterion	Marks	How this video earns evidence
Project understanding and problem analysis	5	Clear railway problem, objectives and justification.
System design and architecture	10	Package/layer diagram, shared state and workflow explanation.
Progress demonstration	40	Live execution of trains, booking, queue, stack, network, hashing/set and tested trees.
Application and justification of structures/algorithms	20	Every member explains why their assigned structure suits the feature.
Individual contribution and technical explanation	20	All six members appear, explain assigned code, complexity, challenge and solution.
Professionalism, organization and clarity	5	Timed transitions, readable screen, rehearsed speech and clear evidence.

2. Mandatory six-member allocation

Use the allocation specified by the assignment brief. Replace “Member 1” etc. with actual names on the title slide.

Member	Primary Week 10 ownership	Relevant project files	Later sorting ownership
1	Array, linked list, linear search, architecture introduction	`StationArray.java`, `TrainLinkedList.java`, `Train.java`, `RailwaySystem.java`	Linear search analysis
2	Stack, queue and booking/waiting-list integration	`ActionStack.java`, `WaitingListQueue.java`, `BookingService.java`, `BookingAction.java`	Bubble Sort
3	Binary Search Tree	`TrainBST.java`, `Train.java`	Selection Sort
4	AVL Tree and rotations	`TrainAVLTree.java`, `Train.java`	Insertion Sort
5	Graph representation, BFS and DFS	`RailwayGraph.java`, `Station.java`, `RailwayNetworkMenu.java`	Merge Sort
6	Hash table and Set ADT	`UserHashTable.java`, `TrainIdSet.java`, `AuthenticationService.java`	Quick Sort

What every member must understand

- The program starts in `Application.main`, creates one `RailwaySystem`, seeds data, and opens `MainMenu`.
- `model` holds railway objects; `structure` holds data structures; `service` holds business rules; `menu` handles console interaction; `railway` coordinates the application.
- The linked list stores complete trains; the Set stores only unique IDs; bookings keep references to the same Train objects.
- A full train sends a user to the FIFO waiting queue. Cancellation frees a seat and assigns it to the earliest matching queued user.
- Stack is LIFO; queue is FIFO; hashing gives average O(1); BST can degrade to O(n); AVL stays O(log n); BFS/DFS are O(V+E).
- All data is in memory, passwords are plain text, and current train routes contain only start and destination. These are acceptable progress-stage limitations, not hidden facts.

3. Exact 9:15 video timeline

Time	Speaker	Section	Visual evidence
0:00–0:35	Member 1	Title, problem and objectives	Title slide with all names
0:35–1:05	Member 1	Architecture and workflow	Package diagram / UML
1:05–2:05	Member 1	Array, linked list and linear search	Stations; admin train view/search
2:05–3:20	Member 2	Stack, queue and integrated booking flow	T104 booking, wait list, cancellation, recent actions
3:20–4:15	Member 3	BST	Source + tested insert/search/delete/traversals output
4:15–5:10	Member 4	AVL	Source + imbalance/rotation + traversal output
5:10–6:20	Member 5	Graph, BFS and DFS	Connections; BFS and DFS from CMB
6:20–7:30	Member 6	Hash table and Set ADT	User buckets/collision and duplicate train rejection
7:30–8:20	Members 2 & 6	Testing and evidence summary	Test table / screenshots / compile result
8:20–8:55	Members 3–6	Challenges and solutions	One concise point each
8:55–9:15	Member 1	Future plan and conclusion	Milestone slide

> **Recording rule:** every member must be visibly identifiable and speak. Use webcam tiles or introduce each member on camera before their screen recording. Keep the final export between 8:00 and 10:00.

4. Member 1 script — introduction, architecture, array, linked list

0:00–2:05

Opening and objectives

SHOW / DO THIS ON SCREEN

Show a title slide: project title, module, group number, six names/IDs, assigned Railway domain, and “Week 10 Progress Demonstration”.

SAY THIS

Good [morning/afternoon]. We are presenting our Data Structures-Based Smart Railway Management System. The objective is to manage trains, users, ticket bookings, waiting passengers and railway connections while demonstrating the practical use of the data structures covered up to Week 10. Our system is implemented in Java as a console application. All six members contributed to design, implementation, integration, testing and documentation, with each member taking primary ownership of the component assigned in the brief.

Architecture

SHOW / DO THIS ON SCREEN

Show the UML class diagram or a simple architecture slide:

```
Application → RailwaySystem → Menus → Services → Models / Structures
model: Train, User, Booking, Station
structure: Array, List, Stack, Queue, BST, AVL, Graph, Hash Table, Set
```

SAY THIS

The project uses a layered package structure. The railway package starts and initializes the application. Models represent Train, User, Booking and Station data. Custom structures implement the required data structures. Services contain validation and business rules, while menus handle console input and output. RailwaySystem owns the shared structures so every menu and service operates on the same in-memory objects.

Array

SHOW / DO THIS ON SCREEN

Run: Main menu → Admin → password `admin123` → Railway network → Stations. Briefly show `StationArray.addStation` and its `String[] stations`.

SAY THIS

My first structure is a fixed-size array for predefined station names. A size variable identifies the next unused position. Adding at that position is O(1) when capacity remains. Searching and displaying scan the occupied elements and therefore take O(n). The array is suitable because the configuration has a known maximum capacity and supports simple indexed storage.

Linked list and linear search

SHOW / DO THIS ON SCREEN

Admin → Manage trains → View all. Then Search → enter `T101`. Optionally add `T105`, show it, then delete it so demo data is restored. Display `TrainNode { Train train; TrainNode next; }` in the IDE.

SAY THIS

Complete Train objects are stored in a custom singly linked list. Each node stores one Train and a reference to the next node. This supports a dynamic number of trains without shifting array elements. Adding at the current tail implementation, searching by ID, deleting and displaying take O(n). The search compares each train ID sequentially, so this also demonstrates linear search. Its best case is O(1) when the first train matches and its average and worst cases are O(n).

Member 1 must understand

- `head` points to the first train node; `null` marks the end.
- Deleting the head differs from deleting a later node.
- Array capacity is 10; only `size` positions are occupied.
- Space: station array O(capacity); linked list O(n).
- Challenge/solution: dynamic train records needed flexible storage, so a linked list was used.

5. Member 2 script — stack, queue and booking integration

2:05–3:20

SHOW / DO THIS ON SCREEN

Prepare two accounts, `alice` and `bob`. Train `T104` has one seat and route `Colombo Fort → Jaffna`.

1. Log in as Alice; Book ticket; enter route and `T104`; show successful booking and note its displayed booking ID.
2. Log out; log in as Bob; attempt the same route/train; show that Bob joins the waiting list; select "My waiting list".
3. Log out; log in as Alice; "My bookings"; cancel the displayed booking ID.
4. Log in as Bob; "My bookings" shows automatic allocation; "Recent actions" shows newest-first activity.

SAY THIS

My component integrates a custom stack and queue with ticket booking. The recent-action stack is LIFO: every successful booking, cancellation or automatic queue booking is pushed at the top, so the newest action is displayed first. Push, pop and peek are O(1), while displaying one user's actions scans the stack in O(n).

SAY THIS

The waiting list is a custom linked FIFO queue with front and rear references. When T104 is full, Bob is enqueued at the rear. When Alice cancels, BookingService frees the seat, removes the earliest waiting entry for that train and creates Bob's booking automatically. Standard enqueue and front dequeue are O(1). Because one global queue contains entries for different trains, dequeueForTrain searches for the earliest matching train and is O(n). Duplicate waiting entries are rejected.

Point to these methods

File/method	What to explain
`ActionStack.push`	New node points to old top; top moves to new node.
`WaitingListQueue.enqueue`	Rear insertion; update front when first item is added.
`BookingService.cancel`	Free seat, record cancellation, call automatic allocation.
`assignSeatToNextUser`	Dequeue matching passenger, reserve seat and record action.

Member 2 must understand

- LIFO versus FIFO and why each fits its feature.
- Why both `front` and `rear` are needed for O(1) enqueue.
- Space for both structures is O(n).
- Challenge/solution: a cancelled seat must go to the correct train's earliest waiting user; the queue scans for the first matching train.

6. Member 3 script — Binary Search Tree

3:20–4:15

> **Before recording:** add and test `delete`, `displayPreOrder` and `displayPostOrder`. The rubric explicitly names them. If incomplete, say "currently in progress" and do not show fabricated output.

SHOW / DO THIS ON SCREEN

Show `TrainBST.java` beside a tiny test output. Insert trains in this order: `T20, T10, T30, T25`. Demonstrate:

```
Search T25 → found
Inorder   → T10 T20 T25 T30
Preorder  → T20 T10 T30 T25
Postorder → T10 T25 T30 T20
Delete T20
Inorder   → T10 T25 T30
```

If the exact root/implementation differs after deletion, show the real tested output rather than memorizing an assumed result.

SAY THIS

My component is a Binary Search Tree that indexes Train objects by train ID. Each node contains a Train, a left child and a right child. IDs smaller than the current node go left and larger IDs go right; duplicate IDs are not inserted. Search follows only one branch according to the comparison. Inorder traversal produces sorted train IDs, while preorder visits root-left-right and postorder visits left-right-root.

SAY THIS

Deletion has three cases: a leaf is removed directly; a node with one child is replaced by that child; and a node with two children is replaced by its inorder successor, the smallest node in its right subtree. In a balanced BST, insert, search and delete are O(log n), but sorted insertion can make it skewed, causing O(n) worst-case time. Traversals are O(n), and storage is O(n).

Member 3 must understand

- BST property and case-insensitive ID comparison.
- All three deletion cases and the inorder successor.
- Difference among inorder, preorder and postorder.
- Best search O(1) at root; average O(log n); worst O(n).
- Challenge/solution: duplicate IDs are ignored/rejected to preserve one key per node.

7. Member 4 script — AVL Tree

4:15–5:10

> **Before recording:** add and test AVL deletion plus preorder and postorder. AVL deletion must also update heights and rebalance nodes while recursion returns.

SHOW / DO THIS ON SCREEN

Show `TrainAVLTree.java` and a three-node demonstration:

```
Insert T10, T20, T30
Before balancing: T10 → T20 → T30 (right-heavy)
Balance at T10 = 0 - 2 = -2
Left rotation
New root: T20; children: T10 and T30
Inorder: T10 T20 T30
Search T30 → found
```

Also display a one-slide table: LL → right rotation; RR → left rotation; LR → left then right; RL → right then left.

SAY THIS

My component is an AVL tree, which is a self-balancing Binary Search Tree. Each node additionally stores its height. After insertion or deletion, the balance factor is calculated as left-subtree height minus right-subtree height. A value of minus one, zero or one is balanced. If it becomes greater than one or less than minus one, rotations restore balance.

SAY THIS

In this example, inserting T10, T20 and T30 creates a right-right imbalance with balance factor minus two, so a left rotation makes T20 the root. The four cases are left-left, right-right, left-right and right-left. AVL search, insertion and deletion remain O(log n) in the worst case because the height stays logarithmic. Traversals remain O(n), and storage is O(n).

Member 4 must understand

- Height of null is 0; a new leaf has height 1 in this implementation.
- `height = 1 + max(leftHeight, rightHeight)`.
- How pointers change during left and right rotations.
- Why heights must be updated bottom-up after rotations.
- BST versus AVL: same ordering, but only AVL guarantees logarithmic height.
- Challenge/solution: rotation order is critical; update the old root before the new root.

8. Member 5 script — graph, BFS and DFS

5:10–6:20

SHOW / DO THIS ON SCREEN

Admin or User → Railway network:

1. Select Connections and show the adjacency-list representation.
2. Select BFS; enter `CMB`; capture actual output.
3. Select DFS; enter `CMB`; capture actual output.
4. Show the railway graph slide/diagram with CMB connected to KDY, MTR and JFN, and KDY connected to BDL.

SAY THIS

My component represents the railway network as an undirected graph. Stations are vertices and direct railway connections are edges. An adjacency list is suitable because this network is sparse and it stores only existing connections. Adding a route in both directions represents two-way travel.

SAY THIS

Breadth-First Search uses a queue and visits stations level by level. Starting at CMB, it first visits the direct neighbours before stations farther away. Depth-First Search uses recursion and follows one branch deeply before backtracking. A visited set prevents repeated visits and cycles. Both BFS and DFS take O(V plus E) time. Adjacency-list space is O(V plus E); BFS additionally uses O(V) queue/visited space and recursive DFS can use O(V) call-stack space.

Point to these details

- `stations` maps IDs to Station objects.
- `adjacencyList` maps each station ID to neighbour IDs.
- `addRoute` inserts both directions and rejects invalid/self routes.
- BFS calls `queue.offer` / `poll`; DFS calls `dfsRecursive`.

Member 5 must understand

- Vertex, edge, path, neighbour, adjacency list and visited set.
- BFS is appropriate for shortest paths in an unweighted graph, although the current method demonstrates traversal rather than returning a specific path.
- Challenge/solution: cycles could cause infinite revisits; the visited set prevents them.

9. Member 6 script — hash table and Set ADT

6:20–7:30

SHOW / DO THIS ON SCREEN

1. Register `alice` and `david`. With the current 10-bucket table these names map to the same bucket, providing collision evidence.
2. Admin → Manage users → Display table. Point to the chain such as `[david] → [alice]` in one bucket. Use the actual output.
3. Admin → Manage trains → Display train ID set.
4. Try adding a train using existing ID `T101`; show "A train with that ID already exists."

SAY THIS

My component contains a custom user hash table and a custom hash-based Set ADT. The user hash function normalizes the username, obtains its Java hash code, and uses floor modulus with the bucket-array length to produce a valid index. This supports efficient registration and login lookup.

SAY THIS

Different usernames may produce the same bucket index. We handle collisions using separate chaining: each bucket is the head of a linked list of user nodes. Alice and David demonstrate a real collision in our ten-bucket table. Average insert, retrieve and remove time is O(1), while the worst case is O(n) if many keys form one chain. Space is O(n plus bucket capacity).

SAY THIS

The Train ID Set stores normalized unique IDs rather than complete Train objects. Before a train is added, the service checks the set. An existing T101 is rejected, demonstrating duplicate prevention. Set add, contains and remove are average O(1) and worst O(n) with chaining.

Member 6 must understand

- Hash function, bucket, collision and separate chaining.
- Why `Math.floorMod` avoids a negative array index.
- Why IDs are uppercased and usernames compared case-insensitively.
- Difference between a map/hash table (key → User) and a set (unique IDs only).
- Challenge/solution: collisions are unavoidable; chaining preserves all colliding entries.

10. Testing evidence and closing script

7:30–9:15

Testing evidence slide

SHOW / DO THIS ON SCREEN

Show a compact table with green Pass results. Include screenshots beside it if readable.

Test	Input/action	Expected evidence
Array validation	Add/search known station	Station displayed; duplicate rejected
Linked-list linear search	Search T101 and unknown ID	Found / not found
Queue allocation	Fill T104, queue Bob, cancel Alice	Bob automatically receives seat
Stack order	Book then cancel	Newest action displayed first
BST	Insert/search/delete/traversals	Correct sorted/traversal output
AVL	Insert T10,T20,T30	Rotation; T20 becomes root
Graph	BFS/DFS from CMB	All reachable vertices visited once
Hash collision	Register alice and david	Both retained in one chain
Set duplicate	Add existing T101	Duplicate rejected

SAY THIS

We performed positive, negative and boundary tests across the structures. Examples include valid and invalid searches, duplicate ID rejection, collision-chain retrieval, empty structure behaviour, a full-train waiting list, automatic seat assignment and graph traversal from both valid and invalid station IDs. The complete source compiles successfully, and our preliminary results confirm that the demonstrated operations behave as expected.

Concise challenge statements — one per responsible member

- **Member 1:** "We separated dynamic train records from the fixed station configuration using a linked list and array respectively."
- **Member 2:** "We linked cancellation to the earliest matching waiting entry while preventing duplicate queue entries."
- **Member 3:** "We handled the three BST deletion cases using an inorder successor."
- **Member 4:** "We corrected imbalances by updating heights and applying the appropriate single or double rotation."
- **Member 5:** "We prevented repeated traversal in a cyclic graph using a visited set."
- **Member 6:** "We preserved colliding keys through separate chaining and normalized IDs for case-insensitive uniqueness."

Future-development and conclusion script

SHOW / DO THIS ON SCREEN

Show milestones: sorting algorithms → timing comparison → full tree/menu integration → final tests → report/screenshots → viva rehearsal.

SAY THIS

Before the final submission, Member 2 will implement Bubble Sort, Member 3 Selection Sort, Member 4 Insertion Sort, Member 5 Merge Sort and Member 6 Quick Sort. We will compare execution times and best, average, worst and space complexities using common test data. We will complete any remaining tree integration and operations, run full integration and performance tests, finalize screenshots and documentation, and prepare every member for questions on the complete system. This concludes our Week 10 progress demonstration. Thank you.

11. Complexity master sheet

Component / operation	Best	Average	Worst	Auxiliary/storage space
Station array add at next position	O(1)	O(1)	O(1)	O(capacity)
Station array search	O(1)	O(n)	O(n)	O(1) auxiliary
Linked-list search/delete	O(1)	O(n)	O(n)	O(n) storage
Linked-list add at end (current code)	O(1) empty	O(n)	O(n)	O(1) auxiliary
Stack push/pop/peek	O(1)	O(1)	O(1)	O(n) storage
Queue enqueue/front dequeue	O(1)	O(1)	O(1)	O(n) storage
Queue matching train search/dequeue	O(1)	O(n)	O(n)	O(1) auxiliary
BST search/insert/delete	O(1) search root	O(log n)	O(n)	O(n) storage; recursion up to O(n)
BST traversal	O(n)	O(n)	O(n)	O(h) recursion
AVL search/insert/delete	O(1) search root	O(log n)	O(log n)	O(n) storage; O(log n) recursion
AVL traversal	O(n)	O(n)	O(n)	O(log n) recursion
Hash table / Set add,get,remove	O(1)	O(1)	O(n)	O(n + capacity)
BFS	O(V + E)			O(V) auxiliary; graph O(V+E)
DFS	O(V + E)			O(V) visited/call stack; graph O(V+E)

> **Do not say every operation is O(log n).** Use the current implementation. For example, linked-list tail insertion is O(n) because there is no tail pointer, and `dequeueForTrain` is O(n) because it searches a mixed-train queue.

12. Likely viva questions and safe answers

Question	Strong concise answer
Why use both a linked list and trees for trains?	The linked list is the main dynamic collection and supports sequential display; trees can act as indexes for ordered traversal and faster ID search. In a final integrated design, all indexes should reference the same Train objects.
Why is AVL faster than BST?	It is not always faster for every small input, but it guarantees O(log n) height by rotations. A normal BST can become skewed and reach O(n).
Why use a Set if the linked list can check duplicates?	The linked list duplicate check is O(n). A hash-based Set provides average O(1) membership checks and explicitly models uniqueness.
How are hash collisions handled?	Separate chaining: each bucket points to a linked list, and all keys with that bucket index are retained and compared.
Why does BFS use a queue?	FIFO order processes vertices in discovery order, producing level-by-level traversal.
Why does the waiting queue sometimes take O(n)?	The system has one queue containing multiple train IDs, so assigning a seat searches for the earliest entry for a particular train.
What happens when a duplicate train ID is entered?	TrainService checks TrainIdSet and the linked list; it rejects the addition with an error message.
What is the current biggest limitation?	Data is in memory and intermediate route stops are not modelled. Also describe the truthful current tree integration status.
What will be done after Week 10?	Implement and benchmark the assigned sorting algorithms, complete remaining integration, full testing, complexity report, documentation and viva preparation.

Every member's 30-second architecture answer

> **SAY THIS**
>
> Application creates RailwaySystem and opens MainMenu. RailwaySystem owns the shared structures. Models hold railway data, custom structure classes implement the algorithms, services apply validation and business rules, and menus handle user interaction. For example, UserMenu calls BookingService, which updates the Train, User bookings, ActionStack and WaitingListQueue using the same shared objects.

13. Recording preparation checklist

Technical readiness — complete before recording

- ✓ Compile all source using the installed JDK and capture a clean successful result.
- ✓ Confirm the console font is at least 16–18 px and usernames/booking IDs are readable.
- **Required check:** complete/test BST and AVL delete, preorder and postorder, or label them honestly as incomplete.
- **Required check:** prepare a repeatable tree demonstration or screenshots from actual output.
- Reset/restart the program before the final take so T104 has one available seat.
- Create Alice and Bob during the demo, or pre-create them and clearly explain the setup.
- Write down Alice's generated booking ID before cancellation.
- Test the `alice` / `david` collision immediately before recording; show actual bucket output.
- Prepare the UML/architecture, graph, complexity and future-plan slides.
- Keep backup screenshots in case a live step fails.

Presentation quality

- Every member says their name, responsibility, implementation, complexity, challenge and solution.
- Do not read code line by line. Point to one key method and explain its logic.
- Do not speed through output; pause for two seconds after each result.
- Remove notifications, hide unrelated windows and avoid showing passwords other than the supplied demo admin password.
- Use one narrator for transitions: "Now Member 4 will demonstrate AVL balancing."
- Rehearse once with a stopwatch; target 9:00–9:30.
- Export at 1080p where possible and verify audio for all six members.

Evidence folder checklist

- Successful compilation screenshot
- Array/station display
- Linked-list add/search/delete
- Stack recent-action order
- Queue automatic seat assignment
- BST operations and traversals
- AVL rotation and operations
- Graph representation, BFS and DFS
- Hash collision chain
- Set duplicate rejection
- Test case table with expected/actual/pass

14. Final “do not lose marks” list

1. **All six members must appear and actively speak.**
2. **Show actual running evidence, not slides alone.** Progress demonstration carries 40 marks.
3. **Explain why each structure was selected**, not only what it is.
4. **State real complexity from this code**, including average and worst cases.
5. **Show preliminary testing**: successful, invalid, duplicate and empty/boundary behaviours.
6. **Be honest about status.** Never describe an unimplemented tree operation as completed.
7. **Keep sorting in the future plan** for Week 11–12, exactly as the brief permits.
8. **Stay within 8–10 minutes.**
9. **Every member must understand the overall architecture**, because the viva can cover any component.
10. **Use your own natural words after learning this script.** The assignment warns that inability to explain submitted work can result in deductions.

> **Best final message to the marker:** the system does not merely contain isolated data-structure classes. The linked list, stack, queue, hash table, set and graph support visible railway features, while the tree demonstration shows ordered and balanced indexing. The group understands the benefits, trade-offs, complexity, testing and remaining development plan.

Prepared from the local CCS2300 assignment brief and the RailwayProjectStructured source as of 27 July 2026. Verify the final source state before recording.